

Paradigmas de Linguagem

1. Dada uma lista, retorne seu último elemento.

? – ultimo(X, [a,b,c,d])

X = d

Em Prolog, uma lista pode ser dividida em:

- **Cabeça (Head):** primeiro elemento
- **Cauda (Tail):** o resto da lista

Notação: [Cabeça | Cauda]

[a, b, c, d] → Cabeça = a, Cauda = [b, c, d]

[_ | Cauda] → a lista dividida: _ é a cabeça (ignorada), Cauda é o resto

Raciocínio:

1. **Caso base:** Se a lista tem apenas UM elemento, esse elemento é o último
2. **Caso recursivo:** Se a lista tem mais elementos, ignore o primeiro e procure o último no resto

Solução:

% Caso base: lista com um único elemento

```
ultimo(X, [X]).
```

% Caso recursivo: ignora a cabeça, busca na cauda

```
ultimo(X, [_|Cauda]) :- ultimo(X, Cauda).
```

2. Dada uma lista, retorne seu k-ésimo elemento.

? – esimo(X, [a,b,c,d,e,f], 3)

X = c

O índice começa em 1 (não em 0). (a = 1, b = 2, c = 3)

Raciocínio:

1. **Caso base:** Se K = 1, retorna a cabeça da lista
2. **Caso recursivo:** Decrementa K e avança para o próximo element

Solução:

% Caso base: índice 1 retorna a cabeça

```
esimo(X, [X|_], 1).
```

% Caso recursivo: joga fora um elemento e diminui K em 1, até K chegar em 1

```
esimo(X, [_|Cauda], K) :-
    K > 1,
    K1 is K - 1,
    esimo(X, Cauda, K1).
```

3. Dada uma lista, retorne a lista invertida.

?- inversa(X, [a,b,c,d])

X = [d,c,b,a]

Raciocínio:

1. **Caso base:** Lista vazia invertida é lista vazia
2. **Caso recursivo:** Inverte a cauda e coloca a cabeça no final

Explicação:

- H = cabeça (primeiro elemento)
- T = cauda (resto da lista)
- inversa(T, TInvertida) = inverte a cauda primeiro
- append(TInvertida, [H], Invertida) = coloca H no final

Solução:

```
% Caso base: lista vazia invertida é lista vazia
```

```
inversa([], []).
```

```
% Caso recursivo: inverte a cauda e coloca a cabeça no final
```

```
inversa([H|T], Invertida) :-
```

```
    inversa(T, TInvertida),
```

```
    append(TInvertida, [H], Invertida).
```

4. Dada uma lista, testa se é um palíndromo.

?- palindromo(X, [a,b,b,a])

X = true

Palíndromo é uma lista que é igual quando invertida.

- [a,b,b,a] invertida = [a,b,b,a] → é palíndromo
- [a,b,c] invertida = [c,b,a] → não é palíndromo

Raciocínio:

Reutilizo a função inversa e comparar a lista com sua inversa.

Solução:

```
% Usa a função inversa do exercício anterior
```

```

inversa([], []).
inversa([H|T], Invertida) :-
    inversa(T, TInvertida),
    append(TInvertida, [H], Invertida).

% É palíndromo se a lista for igual à sua inversa

palindromo(true, Lista) :- inversa(Lista, Lista).
palindromo(false, Lista) :- \+ inversa(Lista, Lista).

```

5. Dada uma lista estruturada, achata seus elementos numa lista simples.
?- achata(X, [a,[b,[c,d],e]])
X = [a,b,c,d,e]

Achatar: Transformar uma lista com sublistas aninhadas em uma lista simples (sem sublistas).

Raciocínio:

Três casos possíveis:

1. Lista vazia → resultado vazio
2. Cabeça é uma lista → achata a cabeça, achata a cauda, junta tudo
3. Cabeça é um átomo → mantém o átomo e achata o resto

Funções importantes:

- `is_list(H)` → verifica se H é uma lista
- `\+ is_list(H)` → verifica se H não é uma lista (é átomo)
- `append(A, B, C)` → junta lista A com B, resultado em C

Solução:

% Caso base: lista vazia

```
achata([], []).
```

% Caso: cabeça é uma lista - achata ela também

```

achata(Resultado, [H|T]) :-
    is_list(H),
    achata(HAchatado, H),
    achata(TAchatado, T),
    append(HAchatado, TAchatado, Resultado).

```

% Caso: cabeça é um átomo - mantém e continua

```

achata([H|TAchatado], [H|T]) :-
    \+ is_list(H),
    achata(TAchatado, T).

```

6. Dada uma lista, elimine elementos duplicados consecutivos.

?- `comprime(X, [a,a,a,a,b,b,c,a,a,d,e,e,e])`

`X = [a,b,c,a,d,e]`

Atenção: Remover apenas duplicados consecutivos.

Raciocínio:

1. Lista vazia → vazia
2. Lista com um elemento → mantém
3. Dois primeiros iguais → ignora o primeiro, continua
4. Dois primeiros diferentes → mantém o primeiro, continua

Explicação:

- `[H,H|T]` → os dois primeiros são iguais (mesmo H)
- `[H1,H2|T]` → os dois primeiros são diferentes
- `H1 \= H2` → H1 é diferente de H2

Solução:

% Caso base: lista vazia

```
comprime([], []).
```

% Caso base: lista com um elemento

```
comprime([X], [X]).
```

% Caso: dois primeiros iguais - ignora o primeiro

```
comprime([H,H|T], Resultado) :-  
    comprime([H|T], Resultado).
```

% Caso: dois primeiros diferentes - mantém o primeiro

```
comprime([H1,H2|T], [H1|Resultado]) :-  
    H1 \= H2,  
    comprime([H2|T], Resultado).
```

7. Dada uma lista, empacota elementos duplicados consecutivos em sublistas.

?- `empacota(X, [a,a,a,a,b,b,c,a,a,d,e,e,e])`

`X = [[a,a,a,a],[b,b],[c],[a,a],[d],[e,e,e]]`

Agrupar elementos iguais consecutivos em sublistas.

Raciocínio:

1. Pega o primeiro elemento

2. Coleta todos os iguais consecutivos em um grupo
3. Continua com o restante

Usamos um predicado auxiliar `divide` para separar iguais dos diferentes.

Explicação do `divide`:

- `divide(a, [a,a,b,c], Xs, Resto) → Xs = [a,a], Resto = [b,c]`
- Coleta os as consecutivos e retorna o resto

Solução:

```
% Caso base: lista vazia
```

```
empacota([], []).
```

```
% Caso recursivo: cria grupo com X e seus iguais consecutivos
```

```
empacota([X|Xs]|Resto), [X|T]) :-
```

```
    divide(X, T, Xs, Restante),
```

```
    empacota(Resto, Restante).
```

```
% divide(Elemento, Lista, IguaisConsecutivos, Restante)
```

```
% Caso base: lista vazia
```

```
divide(_, [], [], []).
```

```
% Caso: elemento igual - adiciona ao grupo
```

```
divide(X, [X|T], [X|Xs], Resto) :-
```

```
    divide(X, T, Xs, Resto).
```

```
% Caso: elemento diferente - para de agrupar
```

```
divide(X, [Y|T], [], [Y|T]) :-
```

```
    X \= Y.
```

8. Dada uma lista, comprime seus elementos no formato `[N,E]`, onde `N` é o número de duplicatas do elemento `E`, mas elementos sem duplicata são copiados simplesmente.

?- `codifica(X, [a,a,a,a,b,b,c,a,a,d,e,e,e])`

`X = [[4,a],[2,b],c,[2,a],d,[3,e]]`

Elementos repetidos → `[quantidade, elemento]`

Elementos únicos → mantém simples

Raciocínio:

1. Usar `empacota` (exercício 7) para criar grupos
2. Transformar cada grupo:

- Grupo com 1 elemento → elemento simples
- Grupo com N elementos → [N, elemento]

Explicação:

- `length([X|Xs], N)` → conta quantos elementos tem no grupo
- Se $N > 1$ → retorna [N, X]
- Se $N = 1$ → retorna só X

Solução:

% Predicado principal

```
codifica(Resultado, Lista) :-
    empacota(Grupos, Lista),
    transforma(Resultado, Grupos).
```

% --- Transforma grupos no formato desejado ---

% Caso base

```
transforma([], []).
```

% Grupo com mais de 1 elemento - [N, E]

```
transforma([N,X|Resto], [[X|Xs]|Grupos]) :-
    length([X|Xs], N),
    N > 1,
    transforma(Resto, Grupos).
```

% Grupo com 1 elemento - mantém simples

```
transforma([X|Resto], [[X]|Grupos]) :-
    transforma(Resto, Grupos).
```

% --- Empacota do exercício 7 ---

```
empacota([], []).
empacota([X|Xs|Resto], [X|T]) :-
    divide(X, T, Xs, Restante),
    empacota(Resto, Restante).
divide(_, [], [], []).
divide(X, [X|T], [X|Xs], Resto) :-
    divide(X, T, Xs, Resto).
divide(X, [Y|T], [], [Y|T]) :-
    X \= Y.
```

9. Dada uma lista e um ponto de corte, divide a lista em duas partes.
 ?- `divide([a,b,c,d,e,f,g,h,i,j,k], 3, L1, L2)`

L1 = [a,b,c]
L2 = [d,e,f,g,h,i,j,k]

Pegar os primeiros N elementos para L1, o restante vai para L2.

Raciocínio:

1. Caso base: Se $N = 0$, L1 é vazia e L2 é a lista inteira
2. Caso recursivo: Pega a cabeça para L1, decrementa N, continua

Explicação:

- `divide(Lista, 0, [], Lista)` → quando N chega a 0, para de cortar
- `[H|L1]` → adiciona H na primeira lista
- `N1 is N - 1` → decrementa o contador

Solução:

% Caso base: N = 0, L1 vazia, L2 é o restante

`divide(Lista, 0, [], Lista).`

% Caso recursivo: pega elemento para L1, decrementa N

`divide([H|T], N, [H|L1], L2) :-`

`N > 0,`

`N1 is N - 1,`

`divide(T, N1, L1, L2).`

10. Dada uma lista e dois pontos de corte, faz uma cópia da sublista.

?- sublista([a,b,c,d,e,f,g,h,i,j,k], 3, 7, L)

L = [c,d,e,f,g]

Extrair elementos da posição 3 até a posição 7 (índice começa em 1).

Raciocínio:

Reutilizo o divide do exercício 9:

1. Pular os primeiros (Início - 1) elementos
2. Pegar os próximos (Fim - Início + 1) elementos

Explicação:

- `I1 is Inicio - 1` → quantos pular ($3-1 = 2$, pula a,b)
- `Tamanho is Fim - Inicio + 1` → quantos pegar ($7-3+1 = 5$ elementos)
- Primeiro divide → ignora os primeiros, Resto = [c,d,e,f,g,h,i,j,k]
- Segundo divide → pega 5 elementos, L = [c,d,e,f,g]

Solução:

```
sublista(Lista, Inicio, Fim, L) :-
```

```
    Il is Inicio - 1,
```

```
    Tamanho is Fim - Inicio + 1,
```

```
    divide(Lista, Il, _, Resto),
```

```
    divide(Resto, Tamanho, L, _).
```

```
% --- Divide do exercício 9 ---
```

```
divide(Lista, 0, [], Lista).
```

```
divide([H|T], N, [H|L1], L2) :-
```

```
    N > 0,
```

```
    N1 is N - 1,
```

```
    divide(T, N1, L1, L2).
```

11. Dada uma lista e um inteiro N, rotaciona a lista N posições.

?- rotaciona([a,b,c,d,e,f,g,h], 3, L)

L = [d,e,f,g,h,a,b,c]

?- rotaciona([a,b,c,d,e,f,g,h], -2, L)

L = [g,h,a,b,c,d,e,f]

N positivo → move os primeiros N elementos para o final

N negativo → move os últimos |N| elementos para o início

Raciocínio:

Reutilizo o divide do exercício 9:

1. Divide a lista no ponto de corte
2. Junta na ordem inversa (L2 + L1)

Para N negativo: rotação de -2 em lista de 8 = rotação de 6

Explicação:

- divide(Lista, 3, L1, L2) → L1 = [a,b,c], L2 = [d,e,f,g,h]
- append(L2, L1, L) → junta [d,e,f,g,h] + [a,b,c] = [d,e,f,g,h,a,b,c]
- Para N = -2: Tamanho + N = 8 + (-2) = 6, rotaciona 6 posições

Solução:

```
% Caso: N positivo ou zero
```

```
rotaciona(Lista, N, L) :-
```

```
    N >= 0,
```

```
    divide(Lista, N, L1, L2),
```

```
    append(L2, L1, L).
```



```
% Caso: N negativo
```

```
rotaciona(Lista, N, L) :-
```

```
    N < 0,
```

```
    length(Lista, Tamanho),
```

```
    N1 is Tamanho + N,
```

```
    rotaciona(Lista, N1, L).
```

```
% --- Divide do exercício 9 ---
```

```
divide(Lista, 0, [], Lista).
```

```
divide([H|T], N, [H|L1], L2) :-
```

```
    N > 0,
```

```
    N1 is N - 1,
```

```
    divide(T, N1, L1, L2).
```

12. Dada uma lista e um inteiro K, remove o K-ésimo elemento da lista.

?- remove(X, [a,b,c,d,e,f,g,h], 3, L)

X = c

L = [a,b,d,e,f,g,h]

Remover o elemento na posição K e retornar tanto o elemento removido (X) quanto a lista resultante (L).

Raciocínio:

1. Caso base: Se K = 1, remove a cabeça
2. Caso recursivo: Mantém a cabeça, decrementa K, continua

Explicação:

- remove(X, [X|T], 1, T) → quando K=1, X é a cabeça e T é o resto
- [H|L] → mantém H no resultado e continua removendo em L

Solução:

```
% Caso base: K = 1, remove a cabeça
```

```
remove(X, [X|T], 1, T).
```

```
% Caso recursivo: mantém a cabeça, decrementa K
```

```
remove(X, [H|T], K, [H|L]) :-
```

```
    K > 1,
```

```
    K1 is K - 1,
```

```
    remove(X, T, K1, L).
```

13. Dada uma lista e dois inteiros, K e N, insere N na K-ésima posição da lista.

?- insere([a,b,c,d,e,f,g,h], alfa, 2, L)

```
L = [a,alfa,b,c,d,e,f,g,h]
```

Inserir o elemento na posição K (empurra os demais para frente).

Raciocínio:

1. Caso base: Se $K = 1$, insere no início
2. Caso recursivo: Mantém a cabeça, decrementa K, continua

Explicação:

- `insere(Lista, N, 1, [N|Lista])` → quando $K=1$, coloca N na frente da lista
- `[H|L]` → mantém H no resultado e continua inserindo em L

Solução:

```
% Caso base: K = 1, insere no início
```

```
insere(Lista, N, 1, [N|Lista]).
```

```
% Caso recursivo: mantém a cabeça, decrementa K
```

```
insere([H|T], N, K, [H|L]) :-
```

```
    K > 1,
```

```
    K1 is K - 1,
```

```
    insere(T, N, K1, L).
```

14. Cria uma lista contendo os inteiros de um intervalo.

?- `range(4, 9, L)`

`L = [4,5,6,7,8,9]`

Gerar uma lista com todos os inteiros de Início até Fim (inclusive).

Raciocínio:

1. Caso base: Se Início = Fim, retorna [Fim]
2. Caso recursivo: Adiciona Início à lista, incrementa, continua

Explicação:

- `range(N, N, [N])` → quando início = fim, lista só com esse número
- `[Início|L]` → coloca Início na lista e continua
- `Início1 is Início + 1` → incrementa para o próximo número

Solução:

```
% Caso base: início = fim
```

```
range(N, N, [N]).
```

```
% Caso recursivo: adiciona início, incrementa
```

```
range(Início, Fim, [Início|L]) :-
```

```
Inicio < Fim,
Iniciol is Inicio + 1,
range(Iniciol, Fim, L).
```

15. Exiba a contagem regressiva de N até 0. ?– contagem(N)

Raciocínio:

- Imprimir na tela
- Caso base: $N = 0 \rightarrow$ imprime 0 e termina
- Caso recursivo: imprime N, decrementa, chama recursivamente

Solução:

```
% Caso base: chegou em 0
contagem(0) :-
    write(0), nl.

% Caso recursivo: imprime N e continua
contagem(N) :-
    N > 0,
    write(N), nl,
    N1 is N - 1,
    contagem(N1).
```

16. Retorne em F o fatorial de N. ?– fatorial(N, F)

Raciocínio:

- Caso base: fatorial de 0 é 1
- Caso recursivo: $N! = N \times (N-1)!$

Solução:

```
% Caso base:  $0! = 1$ 
fatorial(0, 1).

% Caso recursivo:  $N! = N * (N-1)!$ 
fatorial(N, F) :-
    N > 0,
    N1 is N - 1,
    fatorial(N1, F1),
    F is N * F1.
```

17. Retorne o máximo de uma lista. ?– max_lista(Lista, Max)

Raciocínio:

- Caso base: lista com um elemento → o máximo é ele mesmo
- Caso recursivo: compara a cabeça com o máximo da cauda

Solução:

% Caso base: lista com um elemento

```
max_lista([X], X).
```

% Caso recursivo: compara cabeça com máximo da cauda

```
max_lista([H|T], Max) :-
```

```
    max_lista(T, MaxT),
```

```
    (H > MaxT -> Max = H ; Max = MaxT).
```

18. Gere a combinação de K objetos distintos selecionados da lista de N elementos.

?- combinacao(3, [a,b,c,d,e,f], L)

L = [a,b,c];

L = [a,b,d];

L = [a,b,e]; continue...

Raciocínio:

- Caso base: combinação de 0 elementos → lista vazia
- Caso recursivo (duas escolhas):
 - a. Pega a cabeça → precisa de K-1 elementos da cauda
 - b. Não pega a cabeça → precisa de K elementos da cauda

O backtracking do Prolog gera todas as combinações!

Solução:

% Caso base: combinação de 0 elementos é lista vazia

```
combinacao(0, _, []).
```

% Caso recursivo 1: pega a cabeça

```
combinacao(K, [H|T], [H|Comb]) :-
```

```
    K > 0,
```

```
    K1 is K - 1,
```

```
    combinacao(K1, T, Comb).
```

% Caso recursivo 2: não pega a cabeça

```
combinacao(K, [_|T], Comb) :-
```

```
    K > 0,
```

```
    combinacao(K, T, Comb).
```

19. Gere a lista L contendo N inteiros aleatórios sorteados no intervalo de A a B.

```
?- sorteioSacola(5,1,3,L)
L = [1,3,2,1,2]
```

Raciocínio:

- Caso base: $N = 0 \rightarrow$ lista vazia
- Caso recursivo: sorteia um número entre A e B, adiciona à lista, repete N-1 vezes
- Usar `random_between(A, B, X)` do SWI-Prolog

Solução:

```
% Caso base: 0 elementos = lista vazia
```

```
sorteioSacola(0, _, _, []).
```

```
% Caso recursivo: sorteia um número e continua
```

```
sorteioSacola(N, A, B, [X|L]) :-
```

```
    N > 0,
```

```
    random_between(A, B, X),
```

```
    N1 is N - 1,
```

```
    sorteioSacola(N1, A, B, L).
```

20. Determine se um número inteiro é primo.

```
?- primo(17)
```

```
yes
```

Raciocínio:

- Um número é primo se $N > 1$ e não tem divisores entre 2 e $\sqrt{N}$
- Usamos um predicado auxiliar para testar divisores

Solução:

```
% N é primo se N > 1 e não tem divisores
```

```
primo(N) :-
```

```
    N > 1,
```

```
    N1 is N - 1,
```

```
    nao_tem_divisor(N, N1).
```

```
% Caso base: chegou em 1, não achou divisor
```

```
nao_tem_divisor(_, 1).
```

```
% Caso recursivo: D não divide N, testa D-1
```

```
nao_tem_divisor(N, D) :-
```

```
    D > 1,
```

```
    N mod D =\= 0,
```

```
D1 is D - 1,
nao_tem_divisor(N, D1).
```

21. **Determine o maior divisor comum de dois números inteiros A e B.**
?– gcd(36,63,G)
G = 9

Raciocínio (Algoritmo de Euclides):

- Caso base: $\text{gcd}(A, 0) = A$
- Caso recursivo: $\text{gcd}(A, B) = \text{gcd}(B, A \bmod B)$

Solução:

```
% Caso base: gcd(A, 0) = A
gcd(A, 0, A).

% Caso recursivo: gcd(A, B) = gcd(B, A mod B)
gcd(A, B, G) :-
    B > 0,
    R is A mod B,
    gcd(B, R, G).
```

22. **Construa uma lista crescente com os fatores primos de um inteiro positivo A.**
?– fatora(315,L)
L = [3,3,5,7]

Raciocínio:

- Começamos testando divisão pelo menor primo (2)
- Se N é divisível por F → adiciona F à lista, continua com N/F
- Se não é divisível → tenta F+1
- Para quando N = 1

Solução:

```
% Inicia fatora  o a partir de 2
fatora(N, L) :-
    N > 1,
    fatora(N, 2, L).

% Caso base: N = 1, n  o h   mais fatores
fatora(1, _, []).

% Caso 1: F divide N → adiciona F e continua com N/F
fatora(N, F, [F|L]) :-
```

```

N > 1,
N mod F == 0,
N1 is N // F,
fatora(N1, F, L).

% Caso 2: F não divide N → tenta próximo fator
fatora(N, F, L) :-
    N > 1,
    N mod F \= 0,
    F1 is F + 1,
    fatora(N, F1, L).

```

23. Crie um predicado que, para uma lista dada, retorne essa lista ordenada, pelo método merge-sort.
 ?- mergesort([23,4,5,13,4], S)
 S = [4,4,5,13,23]

Raciocínio (Dividir e Conquistar):

1. Divide a lista ao meio
2. Ordena cada metade recursivamente
3. Combina (merge) as duas metades ordenadas

```

Solução:

% Caso base: lista vazia ou com 1 elemento
mergesort([], []).
mergesort([X], [X]).

% Caso recursivo: divide, ordena, combina
mergesort(Lista, Ordenada) :-
    Lista = [_, _], % pelo menos 2 elementos
    divide(Lista, L1, L2),
    mergesort(L1, O1),
    mergesort(L2, O2),
    merge(O1, O2, Ordenada).

% Divide lista em duas metades (alternando elementos)
divide([], [], []).
divide([X], [X], []).
divide([X,Y|T], [X|T1], [Y|T2]) :-
    divide(T, T1, T2).

% Merge: combina duas listas ordenadas

```

```
merge([], L, L).
merge(L, [], L).
merge([H1|T1], [H2|T2], [H1|M]) :-
    H1 <= H2,
    merge(T1, [H2|T2], M).
merge([H1|T1], [H2|T2], [H2|M]) :-
    H1 > H2,
    merge([H1|T1], T2, M).
```

24. Crie um predicado que, para uma lista dada, retorne essa lista ordenada pelo método quick-sort.
 ?- quicksort([23,4,5,13,4], S)
 S = [4,4,5,13,23]

Raciocínio:

1. Escolhe o pivô (primeiro elemento)
2. Particiona: menores à esquerda, maiores à direita
3. Ordena cada partição recursivamente
4. Concatena: esquerda + [pivô] + direita

Solução:

% Caso base: lista vazia

```
quicksort([], []).
```

% Caso recursivo: particiona, ordena, concatena

```
quicksort([Pivo|T], Ordenada) :-
    particiona(Pivo, T, Menores, Maiores),
    quicksort(Menores, OrdMenores),
    quicksort(Maiores, OrdMaiores),
    append(OrdMenores, [Pivo|OrdMaiores], Ordenada).
```

% Particiona lista em menores e maiores que o pivô

```
particiona(_, [], [], []).
particiona(Pivo, [H|T], [H|Menores], Maiores) :-
    H <= Pivo,
    particiona(Pivo, T, Menores, Maiores).
particiona(Pivo, [H|T], Menores, [H|Maiores]) :-
    H > Pivo,
    particiona(Pivo, T, Menores, Maiores).
```


25. Crie um predicado que, para uma lista dada, retorne essa lista ordenada pelo método bubble-sort.
?- bubblesort([23,4,5,13,4], S)
S = [4,4,5,13,23]

Raciocínio:

1. Faz uma passada pela lista, trocando elementos adjacentes fora de ordem
2. Se houve troca → repete o processo
3. Se não houve troca → lista ordenada

Solução:

% Bubble sort: faz passadas até não haver trocas

```
bubblesort(Lista, Ordenada) :-  
    passada(Lista, L1, Trocou),  
    (Trocou == sim  
    -> bubblesort(L1, Ordenada)  
    ; Ordenada = L1).
```

% Passada: percorre lista trocando adjacentes fora de ordem

```
passada([], [], nao).  
passada([X], [X], nao).  
passada([X,Y|T], [X|R], Trocou) :-  
    X <= Y,  
    passada([Y|T], R, Trocou).  
passada([X,Y|T], [Y|R], sim) :-  
    X > Y,  
    passada([X|T], R, _).
```

26. Crie uma base de conhecimento para representar livros, autores e editoras.

Use fatos no estilo:

livro(Titulo, Autor, Editora, Ano).

autor(Autor, Nacionalidade).

Implemente predicados para:

- descobrir todos os livros de um autor;
- descobrir todos os autores de uma editora;
- verificar se dois livros são da mesma editora.

Solução:

```
% Fatos: livro(Titulo, Autor, Editora, Ano)
```

```
%      autor(Autor, Nacionalidade)
```

```
% === BASE DE CONHECIMENTO ===
```

```
% Livros
```

```
    livro('Dom Casmurro', 'Machado de Assis', 'Garnier', 1899).
```

```
    livro('Memórias Póstumas', 'Machado de Assis', 'Tipografia Nacional', 1881).
```

```
    livro('O Cortiço', 'Aluísio Azevedo', 'Garnier', 1890).
```

```
    livro('Grande Sertão', 'Guimarães Rosa', 'José Olympio', 1956).
```

```
    livro('Vidas Secas', 'Graciliano Ramos', 'José Olympio', 1938).
```

```
    livro('São Bernardo', 'Graciliano Ramos', 'Ariel', 1934).
```

```
% Autores
```

```
    autor('Machado de Assis', brasileiro).
```

```
    autor('Aluísio Azevedo', brasileiro).
```

```
    autor('Guimarães Rosa', brasileiro).
```

```
    autor('Graciliano Ramos', brasileiro).
```

```
% === PREDICADOS ===
```

```
% 1. Descobrir todos os livros de um autor
```

```
    livros_do_autor(Autor, Livros) :-
```

```
        findall(Titulo, livro(Titulo, Autor, _, _), Livros).
```

```
% 2. Descobrir todos os autores de uma editora
```

```
    autores_da_editora(Editora, Autores) :-
```

```
        findall(Autor, livro(_, Autor, Editora, _), Lista),
```

```
        list_to_set(Lista, Autores). % remove duplicados
```

```
% 3. Verificar se dois livros são da mesma editora
```

```
    mesma_editora(Livro1, Livro2) :-
```

```
        livro(Livro1, _, Editora, _),
```

```
        livro(Livro2, _, Editora, _),
```

```
        Livro1 \= Livro2.
```

Exemplos de consultas:

```
% 1. Livros de Machado de Assis
```

```
    ?- livros_do_autor('Machado de Assis', L).
```

```
    L = ['Dom Casmurro', 'Memórias Póstumas']
```

```
% 2. Autores da editora José Olympio
```

```
    ?- autores_da_editora('José Olympio', A).
```

```
    A = ['Guimarães Rosa', 'Graciliano Ramos']
```

```
% 3. Dom Casmurro e O Cortiço são da mesma editora?
```

```
    ?- mesma_editora('Dom Casmurro', 'O Cortiço').
```

```
true % ambos da Garnier
```

27. Crie uma base de conhecimento para representar disciplinas, seus pré-requisitos

e o histórico de alunos. Use fatos no estilo:

disciplina(Codigo, Nome).

prerequisito(D1, D2). significando: D1 exige D2.

aprovado(Aluno, Disciplina).

Implemente predicados para:

- **verificar se uma disciplina exige diretamente outra: exige(D1,D2)**
- **verificar se uma disciplina exige outra indiretamente: exige_indiretamente(D1,D2)**
- **verificar se um aluno pode cursar uma disciplina: pode_cursar(Aluno, Disciplina)**

Solução:

```
% Fatos: disciplina(Codigo, Nome)
```

```
%      prerequisito(D1, D2) - D1 exige D2
```

```
%      aprovado(Aluno, Disciplina)
```

```
% === BASE DE CONHECIMENTO ===
```

```
% Disciplinas
```

```
disciplina(calc1, 'Calculo 1').
```

```
disciplina(calc2, 'Calculo 2').
```

```
disciplina(calc3, 'Calculo 3').
```

```
disciplina(prog1, 'Programacao 1').
```

```
disciplina(prog2, 'Programacao 2').
```

```
disciplina(ed, 'Estrutura de Dados').
```

```
disciplina(bd, 'Banco de Dados').
```

```
disciplina(ia, 'Inteligencia Artificial').
```

```
% Pré-requisitos (D1 exige D2)
```

```
prerequisito(calc2, calc1).
```

```
prerequisito(calc3, calc2).
```

```
prerequisito(prog2, prog1).
```

```
prerequisito(ed, prog2).
```

```
prerequisito(bd, ed).
```

```
prerequisito(ia, ed).
```

```
prerequisito(ia, calc2).
```

```

% Histórico de alunos (aprovações)
    aprovado(joao, calc1).
    aprovado(joao, calc2).
    aprovado(joao, prog1).
    aprovado(joao, prog2).
    aprovado(maria, calc1).
    aprovado(maria, prog1).
    aprovado(maria, prog2).
    aprovado(maria, ed).

% === PREDICADOS ===

% 1. Verifica se D1 exige diretamente D2
    exige(D1, D2) :-
        prerequisite(D1, D2).

% 2. Verifica se D1 exige D2 indiretamente (cadeia de pré-requisitos)
    exige_indiretamente(D1, D2) :-
        prerequisite(D1, X),
        (X = D2 ; exige_indiretamente(X, D2)).

% 3. Verifica se aluno pode cursar disciplina
%     (deve ter sido aprovado em TODOS os pré-requisitos)
    pode_cursar(Aluno, Disciplina) :-
        disciplina(Disciplina, _),
        \+ aprovado(Aluno, Disciplina), % ainda não cursou
        findall(Pre, prerequisite(Disciplina, Pre), Prerequisitos),
        todos_aprovados(Aluno, Prerequisitos).

% Auxiliar: verifica se aluno foi aprovado em todas as disciplinas da lista
    todos_aprovados(_, []).
    todos_aprovados(Aluno, [D|Resto]) :-
        aprovado(Aluno, D),
        todos_aprovados(Aluno, Resto).

Exemplos de consultas:

% 1. Calc2 exige diretamente Calc1?
    ?- exige(calc2, calc1).
    true

% 2. Calc3 exige indiretamente Calc1?
    ?- exige_indiretamente(calc3, calc1).
    true % calc3 → calc2 → calc1

% 3. João pode cursar ED?
    ?- pode_cursar(joao, ed).

```

```
true % João tem prog1 e prog2
```

```
% 4. Maria pode cursar IA?
```

```
?- pode_cursar(maria, ia).
```

```
false % Maria não tem calc2
```

28. Crie uma representação, estilo grafo, para que modelar um mapa mostrando cidades e estradas bidirecionais: estrada(Id, Cidade1, Cidade2, Distancia).

Depois defina predicados para:

- **verificar se duas cidades são vizinhas: vizinhas(X,Y)**
- **verificar se duas cidades são conectadas: conectadas(X,Y,Caminho)**

Solução:

```
% Fato: estrada(Id, Cidade1, Cidade2, Distancia)
```

```
% === BASE DE CONHECIMENTO ===
```

```
% Estradas (bidirecionais)
```

```
estrada(e1, fortaleza, caucaia, 20).
```

```
estrada(e2, fortaleza, maracanau, 25).
```

```
estrada(e3, fortaleza, aquiraz, 30).
```

```
estrada(e4, caucaia, maranguape, 35).
```

```
estrada(e5, maracanau, maranguape, 15).
```

```
estrada(e6, maracanau, pacatuba, 20).
```

```
estrada(e7, aquiraz, eusebio, 10).
```

```
estrada(e8, eusebio, fortaleza, 15).
```

```
% === PREDICADOS ===
```

```
% 1. Verifica se duas cidades são vizinhas (estrada direta)
```

```
vizinhas(X, Y) :-
```

```
    estrada(_, X, Y, _).
```

```
vizinhas(X, Y) :-
```

```
    estrada(_, Y, X, _). % bidirecional
```

```
% 2. Verifica se duas cidades são conectadas e retorna o caminho
```

```
conectadas(X, Y, Caminho) :-
```

```
    conectadas(X, Y, [X], Caminho).
```

```
% Auxiliar com lista de visitados (evita ciclos)
```

```
conectadas(X, X, Visitados, Caminho) :-
```

```
    reverse(Visitados, Caminho). % chegou ao destino
```

```
conectadas(X, Y, Visitados, Caminho) :-
```

```
vizinhas(X, Z),
\+ member(Z, Visitados), % não revisitar
conectadas(Z, Y, [Z|Visitados], Caminho).
```

Exemplos de consultas:

% São vizinhas?

```
?- vizinhas(fortaleza, caucaia).
true
```

% Caminho entre duas cidades

```
?- conectadas(fortaleza, pacatuba, C).
C = [fortaleza, maracanaú, pacatuba]
```

% Todos os caminhos possíveis

```
?- findall(C, conectadas(fortaleza, maranguape, C), Todos).
```

29. Problema clássico da coloração de grafos: Crie uma representação para mapas em prolog. Depois crie fatos para representar o mapa dos estados da região sudeste do Brasil.

Feito isso, crie regras e consultas para colorir o mapa com poucas cores de forma que estados que fazem fronteira não recebam a mesma cor.

Solução:

% Teorema das 4 cores: qualquer mapa planar pode ser colorido com 4 cores

% === BASE DE CONHECIMENTO ===

% Fronteiras entre estados (bidirecional)

```
fronteira(sp, mg).
fronteira(sp, rj).
fronteira(mg, rj).
fronteira(mg, es).
fronteira(rj, es).
```

% Fronteira é bidirecional

```
vizinho(X, Y) :- fronteira(X, Y).
vizinho(X, Y) :- fronteira(Y, X).
```

% Cores disponíveis

```
cor(vermelho).
cor(azul).
cor(verde).
cor(amarelo).
```

% === PREDICADO DE COLORAÇÃO ===

% Colorir o mapa: atribui cor a cada estado

```
colorir(SP, MG, RJ, ES) :-
```

```

cor(SP), cor(MG), cor(RJ), cor(ES),
SP \= MG, % SP e MG são vizinhos
SP \= RJ, % SP e RJ são vizinhos
MG \= RJ, % MG e RJ são vizinhos
MG \= ES, % MG e ES são vizinhos
RJ \= ES. % RJ e ES são vizinhos

```

Exemplos de consultas:

% Encontrar UMA coloração válida

```

colorir(SP, MG, RJ, ES).
% SP = vermelho, MG = azul, RJ = verde, ES = vermelho

```

% Ver se duas cidades são vizinhas

```

vizinho(sp, mg).
% true
vizinho(sp, es).
% false (SP não faz fronteira com ES)

```

% Ver TODAS as colorações possíveis

```
colorir(SP, MG, RJ, ES).
```

% Contar quantas colorações existem

```

findall([SP, MG, RJ, ES], colorir(SP, MG, RJ, ES), Todas), length(Todas, N).
% N = 84

```

30. Problema clássico de colocar 8 damas: Resolva em prolog o problema de colocar 8 damas num tabuleiro de xadrez sem que nenhuma ataque a outra.

Regras de ataque:

- Mesma linha ✗
- Mesma coluna ✗
- Mesma diagonal ✗

Representação:

Lista de 8 números onde o índice = coluna, valor = linha

Ex: [4,2,7,3,6,8,5,1] → dama da coluna 1 está na linha 4, coluna 2 na linha 2, etc.

Solução:

% Resolver o problema das N damas

```

damas(N, Solucao) :-
    numlist(1, N, Lista),
    permutacao(Lista, Solucao),

```

```

    seguro(Solucao).

% Gera permutacoes da lista
permutacao([], []).
permutacao(Lista, [H|Perm]) :-
    seleciona(H, Lista, Resto),
    permutacao(Resto, Perm).
seleciona(X, [X|T], T).
seleciona(X, [H|T], [H|R]) :-
    seleciona(X, T, R).

% Verifica se nenhuma dama ataca outra
seguro([]).
seguro([L|Resto]) :-
    nao_ataca(L, Resto, 1),
    seguro(Resto).

% Verifica se dama na linha L nao ataca nenhuma das outras
nao_ataca(_, [], _).
nao_ataca(L, [L2|Resto], Dist) :-
    L \= L2, % linhas diferentes
    abs(L - L2) \= Dist, % diagonais diferentes
    Dist1 is Dist + 1,
    nao_ataca(L, Resto, Dist1).

% Gera lista de 1 a N
numlist(N, N, [N]).
numlist(I, N, [I|Resto]) :-
    I < N,
    I1 is I + 1,
    numlist(I1, N, Resto).

```